

# Data Structures and Algorithms

Balagopal Komarath

2025

## 1 Brute-force search

Brute-force search refers to a class of algorithms that tries each candidate solution one-by-one. It is simple but inefficient. It usually appears as a sub-routine to handle small cases within more efficient algorithms. It can also be used when input sizes are small enough.

We demonstrate brute-force search by counting the number of solutions to the  $n$ -queens problem. In the  $n$ -queens problem, the goal is to find a way, or to find all ways, or to count the number of ways to place  $n$  queens on an  $n \times n$  chessboard such that no queen threatens another. The following is a solution for  $n=4$ :

```
.q..
...q
q...
..q.
```

A brute-force search can solve this problem for small values of  $n$ . In a brute-force search, we try candidate solutions one-by-one. What is a candidate? The simplest definition is that it is a placement of  $n$  queens on an  $n \times n$  board. For  $n=8$ , this gives us  $\binom{64}{8}$  possibilities. A computer can do roughly  $10^8$  operations per second. So going through all such candidates will take at least 45 seconds, assuming one operation per candidate, which is a conservative estimate. If we define candidates as those placements that has one queen per row, then the number of candidates is now only  $8^8$ , which is 45 times less. We may also define candidates by a permutation of  $[0, n)$ , where the  $i^{\text{th}}$  element in the permutation is the column occupied by the queen on the  $i^{\text{th}}$  row. Then, there are only  $8! < 8^8$  candidates. The way we define the candidates can make a huge difference in the running-time.

The following function `nqueens(n)` counts the number of solutions using brute-force search.

```
1 size_t nqueens(size_t n)
2 {
3     size_t c = 0;
4     size_t *queens = malloc(n * sizeof(size_t));
5     assert(queens);
6     first(n, queens);
7     while (true) {
8         if (ok(n, queens))
9             ++c;
10        if (!next(n, queens)) break;
11    }
12    return c;
13 }
```

Observe that in the above function, there is nothing specific to the  $n$ -queens problem. All the details of the problem are encapsulated into the following three functions:

- `first()`, for some fixed ordering of the candidates, writes the first candidate into the array `candidate`.
- `next()` modifies the `candidate` array so that it becomes the next candidate in the fixed ordering. It returns `true` if and only if there was a next candidate. If it returns `false`, then the value in `candidate` should not be used.
- `ok()` returns `true` if and only if the array `candidate` holds a valid solution.

We will now implement these functions for the  $n$ -queens problem. Recall that the `candidate` array stores a permutation of  $[0, n)$ . We will order these permutations lexicographically as follows (for  $n=3$ ): 012, 021, 102, 120, 201, 210. More formally, the ordering is:

**Definition 1.** Let  $p = (p_1, \dots, p_n)$  and  $q = (q_1, \dots, q_n)$  be permutations of some set of  $n$  numbers. They are lexicographically equal if and only if  $p_i = q_i$  for  $1 \leq i \leq n$ . We say that  $p$  is lexicographically less than  $q$  if and only if there is a  $1 \leq j \leq n$  such that  $p_i = q_i$  for all  $i < j$  and  $p_j < q_j$ .

The first permutation is the identity permutation:

```
1 void first(size_t n, size_t *queens)
2 {
3     for (size_t i = 0; i < n; ++i)
4         queens[i] = i;
5 }
```

To find the next permutation, we scan backwards from the end of the array until we see an element that is less than the previous one. Let the position of this element be  $i$ . Then, we swap this element with the rightmost element greater than it in the array. Finally, we reverse the sub-array starting from  $i+1$ . For example:

```
next({1, 5, 2, 3, 0})
      ^ |
      swap
-> {1, 5, 3, 2, 0}
reverse
-> {1, 5, 3, 0, 2}
```

Observe that if we cannot find an element that is less than the previous one while scanning backwards, then the elements are in decreasing order, which is the last permutation. Putting all this together, we obtain the following function:

```
1 bool next(size_t n, size_t *queens)
2 {
3     ssize_t i = n-2;
4     while (i >= 0 && queens[i] >= queens[i+1])
5         --i;
6     if (i < 0)
7         return false;
8     size_t j = n-1;
9     while (queens[j] <= queens[i])
10        --j;
11    swap(&queens[i], &queens[j]);
12    reverse(queens, i+1, n-1);
13    return true;
14 }
```

We now write the function to check validity of the candidate as a solution. By our definition of a candidate, two queens cannot be in the same row or same column. So for each queen, we just need to check whether there is any queen threatening it diagonally.

```
1 bool ok(size_t n, size_t *queens)
2 {
3     for (size_t i = 0; i < n; ++i)
4         for (size_t j = i+1; j < n; ++j) {
5             size_t d;
6             if (queens[i] < queens[j])
7                 d = queens[j]-queens[i];
8             else
9                 d = queens[i]-queens[j];
10            if (d == j-i)
11                return false;
12        }
13    return true;
14 }
```

**Ex. 1** — What is the largest board size  $k$  for which the brute-force search algorithm finishes within five minutes? Determine the wall clock time taken by the `nqueens()` function for  $n \in [2, k]$ .

**Ex. 2** — Modify the  $n$ -queens program to print the first solution. What is the largest board size  $k$  for which you can obtain a solution within five minutes? Determine the number of permutations looked at before finding a solution for  $n \in [2, k]$ .

**Ex. 3** — Write a function:

```
bool le(size_t *p, size_t *q, size_t n);
```

that determines whether or not the permutation  $p$  is lexicographically less than or equal to  $q$ .

**Ex. 4** — Write the function:

```
bool prev(size_t n, size_t *perm);
```

to modify the permutation in `perm` to the previous permutation and return `true` if possible and otherwise return `false`.

**Ex. 5** — Define a candidate as any placement of one queen per row. Rewrite the  $n$ -queens program to use this definition. Do a performance comparison of both solutions.

**Ex. 6** — Describe an algorithm that given two natural numbers  $n$  and  $i$  outputs the  $i^{\text{th}}$  permutation of  $[0, n)$  in lexicographic order. This algorithm should finish within reasonable time for  $n \leq 10^{10}$ .

**Ex. 7** — Describe an algorithm that given a natural number  $n$  and a permutation  $\sigma$  of  $[0, n)$  returns the index of  $\sigma$  in lexicographic ordering.

**Ex. 8** — An ordering of all permutations of  $[0, n)$  is called a *Gray ordering* if two consecutive permutations in the ordering differ at exactly two places. For example, for  $n=3$ :

```
012
021
120
102
201
210
```

is a Gray ordering. Do Gray orderings exist for all  $n$ ? If so, design `first()` and `next()` functions to iterate over them. If not, argue why.

**Ex. 9** — Why does `next()` correctly compute the next permutation?

## 2 Backtracking

Backtracking improves brute-force search by introducing the concept of a *partial candidate* to reduce the number of candidates.

We have seen that brute-force search is too inefficient to solve  $n$ -queens for  $n \geq 13$ . We define a placement of queens on the first  $i$  rows as a partial candidate. Consider the partial candidate 0,1 for  $n=13$ . There are  $11!$  candidates that extend this partial candidate. However, we know that none of them can be a solution because the first two queens threaten each other.

Backtracking is best described using a combinatorial object called a *rooted, ordered tree*. A rooted, ordered tree (hereafter, just referred to as a tree) is a set  $V$  of *nodes* along the function  $\text{children} : V \mapsto V^*$ , where  $V^* = \cup_{i \geq 0} V^i$ . For any  $u \in V$  there is at most one  $v \in V$  such that  $\text{children}(v)$  contains  $u$  and  $u$  appears exactly once in such a sequence. There is exactly one  $r \in V$  such that it does not appear in the image of  $\text{children}$ . This node  $r$  is called the *root* of the tree. A node  $u$  such that  $\text{children}(u) = ()$  is called a *leaf* of the tree.

For  $n=3$ , we can define a tree of partial permutations as follows:

```
children(.) = (0, 1, 2)
children(0) = (01, 02)
children(1) = (10, 12)
children(2) = (20, 21)
children(01) = (012)
children(02) = (021)
children(10) = (102)
children(12) = (120)
children(20) = (201)
children(21) = (210)
children(012) = ()
children(021) = ()
children(102) = ()
children(120) = ()
children(201) = ()
children(210) = ()
```

It is more convenient to use the following representation for a tree of partial permutations of  $[0,3)$ :

```

      .
      0
      01
      012
      02
      021
    1
    10
    102
    12
    120
  2
  20
  201
  21
  210
```

The root node is the empty permutation and is the only node in the first column. It has three children: 0, 1, and 2 that appear in the second column. The node 0 has two children 01 and 12 that appear in the third column and so on.

Backtracking will go through the nodes of the tree in the following order:

```

      .
-> 0
-> 01
<- 0
-> 02
-> 021
<- 02
<- 0
<- .
-> 1
-> 10
<- 1
-> 12
<- 1
<- .
-> 2
-> 20
-> 201
<- 20
<- 2
-> 21
<- 2
<- .
```

The backtracking algorithm performs more work than brute-force search for small  $n$  such as 3 or 4. But for larger values, backtracking performs less work than brute-force search due to the ability to eliminate a large number of candidates without looking at them. The following function implements a backtracking algorithm to count the number of solutions to the  $n$ -queens puzzle:

```
1 size_t nqueens(size_t n)
2 {
3     struct node p;
4     root(n, &p);
5     size_t c = 0;
```

```

6  while (true) {
7      if (leaf(&p)) {
8          if (ok(&p))
9              ++c;
10         if (!up(&p))
11             break;
12     } else {
13         if (!ok(&p)) {
14             if (!up(&p))
15                 break;
16         } else {
17             if (!down(&p))
18                 if (!up(&p))
19                     break;
20         }
21     }
22 }
23 return c;
24 }

```

The function `ok()` returns `false` when the given partial candidate is known to be invalid. The functions `up()` and `down()` move up and down in the tree of partial candidates. They return `false` when this movement is not possible.

Lines 7 to 13 deal with candidate solutions. Lines 13 and 14 implement the backtracking optimization. Observe that failure to move up a tree in lines 10, 14, and 18 indicates that no candidates are left.

The `struct node` represents a node in the tree of partial candidates. The field `k` is the number of elements in the partial permutation. Note that storing only the partial permutation is insufficient. Consider the partial permutation 1 for  $n=3$ . Calling `down()` two times on this node produced the candidates 10 and 12. So the node should also remember the last child that was visited from this node on a `down()` call. The field `last` keeps track of this information. It stores the last element of the partial permutation of the last visited child or `-1` if no such child exists.

```

1 struct node {
2     size_t n;
3     size_t *queens;
4     size_t k;
5     ssize_t last;
6 };

```

To go up from a node, we have to reduce `k` by one and set `last` to the removed element.

```

1 bool up(struct node *p)
2 {
3     if (p->k == 0)
4         return false;
5     p->last = p->queens[p->k-1];
6     --(p->k);
7     return true;
8 }

```

To move down in the tree, we extend the permutation by one more element. This element should be the smallest element greater than `last` that does not appear elsewhere in the partial permutation.

```

1 bool down(struct node *p)
2 {
3     bool *used = malloc(p->n * sizeof(bool));
4     assert(used);
5     for (size_t i = 0; i < p->n; ++i)
6         used[i] = false;
7     for (size_t i = 0; i < p->k; ++i)
8         used[p->queens[i]] = true;
9
10    for (size_t i = p->last+1; i < p->n; ++i)
11        if (!used[i]) {
12            p->queens[p->k] = i;
13            ++(p->k);
14            p->last = -1;
15            free(used);
16            return true;

```

```

17    }
18    free(used);
19    return false;
20 }

```

Lines 3 to 8 determine which elements are already used in the permutation. The `for` loop in line 10 searches for the smallest element larger than `last` that we can use to extend the partial permutation. If there is no such element, then we must have come up to this node from the last child and so we return `false` in line 19.

**Ex. 10** — What is the largest board size  $k$  for which the backtracking algorithm can solve  $n$ -queens within five minutes? Compare these times with that of the brute-force search algorithm.

**Ex. 11** — For both brute-force search and backtracking algorithms, estimate the largest board size for which these algorithms would find the number of solutions within one year from your running-time measurements.

**Ex. 12** — Modify the backtracking algorithm to print the first solution instead. What is the largest board size  $k$  for which the backtracking algorithm can find a solution to the  $n$ -queens puzzle within five minutes? Compare these times with that of the brute-force search algorithm.

**Ex. 13** — Write a SUDOKU solver that uses backtracking search to find a solution.

**Ex. 14** — A Knight's closed tour on a chessboard is a sequence of moves of a Knight that starts and ends at the same square and occupies each square exactly once. Write a program to print a solution given  $n$ ,  $m$ , and the starting square for an  $n \times m$  board.

### 3 Analysis

So far, we have used wall clock time to measure the performance of our algorithms. This is the most direct measure of efficiency. However, it could differ from one machine to another. It also depends on the other programs running on the machine at the time of measurement, called the context.

We can devise measurements to estimate performance that are independent of the machine and context. For the brute-force search, a good measure is the number of times `next()` is called. For the backtracking search, the number of times `up()` and `down()` are called may be used to estimate performance. More generally, we have to determine what to measure after understanding the program.

**Ex. 15** — Measure the above parameters and compare the performance of brute-force search and backtracking. Compare these measurements with actual wall clock time measurements.

### 4 Recursion

Recursion in programming is a technique where we use the function we are writing in the body of the function. The use may be direct, where the function body contains a call to the same function, or indirect, where the function body calls another function that calls this function. This technique quite often yields to simpler, but sometimes slightly inefficient, solutions.

The following `min()` function uses direct recursion.

```

1 int min(const int *a, size_t n)
2 {
3     assert(n >= 1);
4     if (n == 1)
5         return a[0];

```

```

6  int rest = min(a+1, n-1);
7  if (a[0] < rest)
8      return a[0];
9  return rest;
10 }

```

Lines 4 and 5 solves what is called a *base case*. Base cases must be solved without invoking the same function (directly or indirectly). Line 6 to 9 implement the *recursive case*.

The following code uses indirect recursion to implement functions to check whether a natural number is even or odd:

```

1 bool even(int n)
2 {
3     return n == 0 || odd(n-1);
4 }
5
6 bool odd(int n)
7 {
8     return n == 1 || even(n-1);
9 }

```

The above functions are almost as easy to implement without recursion. We will now see a recursive implementation of a function that prints all  $k$ -permutations of a set of  $n$  elements.

```

1 void do_permutations (
2     int *a, size_t n,
3     int *b, size_t k,
4     size_t k1
5 )
6 {
7     if (k1 == k) {
8         print(b, k);
9         return;
10    }
11
12    for (size_t i = 0; i < n; ++i) {
13        swap(&a[0], &a[i]);
14        b[k1] = a[0];
15        do_permutations(a+1, n-1, b, k, k1+1);
16        swap(&a[0], &a[i]);
17    }
18 }
19
20 void permutations (
21     int *a, size_t n,
22     int *b, size_t k
23 )
24 {
25     do_permutations(a, n, b, k, 0);
26 }

```

To generate all  $k$ -permutations, we call the function with the following arguments: The array  $a$  is a set of  $n$  elements and the array  $b$  should have space for storing at least  $k$  elements. We then call the helper function `do_permutations()` to do the actual work. In the `do_permutations()` function, the argument  $k1$  should be the number of elements in  $b$  so far. Lines 7 to 10 is the base case. At this point, we have built one permutation and simply print the array and return. Lines 12 to 17 is the recursive case. We put each element of  $a$  as the next element in the permutation and append each one of the  $(k-1)$ -permutations of the other elements in  $a$  to this element. Since in an array it is difficult to remove an element that is not at the first position, we swap the element to be removed with the first element and then remove the first element.

The following depicts the tree of function calls made to generate all 2-permutations of  $\{0, 1, 2\}$ . We use  $p$  instead of `do_permutations()`. The 2-permutations occurring at the leaves of the tree are printed.

```

1 p({0, 1, 2}, 3, {}, 2, 0)
2 p({1, 2}, 2, {0}, 2, 1)
3 p({2}, 1, {0, 1}, 2, 2)
4 p({1}, 1, {0, 2}, 2, 2)

```

```

5 p({0, 2}, 2, {1}, 2, 1)
6 p({2}, 1, {1, 0}, 2, 1)
7 p({0}, 1, {1, 2}, 2, 2)
8 p({1, 0}, 2, {2}, 2, 1)
9 p({0}, 1, {2, 1}, 2, 2)
10 p({1}, 1, {2, 0}, 2, 2)

```

A recursive function to generate all  $k$ -combinations of an  $n$ -element set is given below:

```

1 void do_combinations (
2     const int *a, size_t n,
3     int *b, size_t k,
4     size_t k1
5 )
6 {
7     if (n < k-k1)
8         return;
9
10    if (k1 == k) {
11        print(b, k);
12        return;
13    }
14
15    b[k1] = a[0];
16    do_combinations(a+1, n-1, b, k, k1+1);
17    do_combinations(a+1, n-1, b, k, k1);
18 }
19
20 void combinations (
21     const int *a, size_t n,
22     int *b, size_t k
23 )
24 {
25     do_combinations(a, n, b, k, 0);
26 }

```

It uses the following recursive formulation: A  $k$ -element set can be constructed by adding the first element to each  $(k-1)$ -element set from the rest of the elements, or by considering all  $k$ -element subsets from the rest of the elements. This formulation also gives rise to the following combinatorial identity:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}, \text{ where } n > k, k > 0.$$

We can implement backtracking search for  $n$ -queens using recursion. Observe that the recursive generation of permutations builds them by starting with the empty permutation and recursively extending them. So we can use it with minimal modifications.

```

1 size_t do_nqueens (
2     size_t n, size_t *queens, size_t k
3 )
4 {
5     if (k == n && ok(queens, k))
6         return 1;
7
8     if (!ok(queens, k))
9         return 0;
10
11    bool *used = malloc(n * sizeof(bool));
12    assert(used);
13    for (size_t i = 0; i < n; ++i)
14        used[i] = false;
15    for (size_t i = 0; i < k; ++i)
16        used[queens[i]] = true;
17
18    size_t c = 0;
19    for (size_t i = 0; i < n; ++i)
20        if (!used[i]) {
21            queens[k] = i;
22            c += do_nqueens(n, queens, k+1);
23        }
24    free(used);
25    return c;
26 }
27
28 size_t nqueens(size_t n)
29 {
30     size_t *queens = malloc(n * sizeof(size_t));
31     assert(queens);

```

```

32 size_t c = do_nqueens(n, queens, 0);
33 free(queens);
34 return c;
35 }

```

Observe that lines 8 and 9 implement backtracking. The `return` statements move up the tree and the recursive call in line 22 move down the tree.

**Ex. 16** — Observe that the recursive function to generate permutations does not generate them in lexicographic order. Write a recursive function to generate permutations in lexicographic order.

**Ex. 17** — Show the tree of calls made to generate all 2-combinations of  $\{0, 1, 2, 3\}$  by the `do_combinations()` function.

**Ex. 18** — Show the tree of calls made by `nqueens()` for  $n=4$ .

**Ex. 19** — Measure the wall clock time for the recursive, backtracking  $n$ -queens solution and compare it with the non-recursive implementation.

**Ex. 20** — Write a recursive function to generate all  $k$ -permutations with replacement for a given  $n$ -element set.

**Ex. 21** — Write a recursive function to generate all  $k$ -combinations with replacement for a given  $n$ -element set.

**Ex. 22** — Write a recursive function to generate all partitions with exactly  $k$  parts for a given  $n$ -element set.

## 5 Searching

The linear search algorithm searches for an element in a sequence of elements. In the following code, we represent the sequence using an array of elements called `haystack`. The element we are looking for is `needle`.

```

1 template <typename T>
2 T* search (
3     T* haystack, size_t n,
4     const T& needle
5 )
6 {
7     for (size_t i = 0; i < n; ++i)
8         if (haystack[i] == needle)
9             return &haystack[i];
10    return nullptr;
11 }

```

Observe that the code is *generic*. That is, it works for any type  $T$ . The only requirement from this type is that it should allow us to check for equality of elements using the operator `==` (See line 8).

Since the algorithm is generic, the wall clock running time will depend on the time taken to compare two elements. We wish to avoid this dependence. So we assume comparison takes some unit time and measure with respect to that unit. Instead of a concrete wall clock time, we study how the performance scales with the input size. Linear search has the following performance characteristics:

**Best Case** The needle is the first element of the haystack. The running time is independent of the size of the haystack. In other words, the running time is bounded by a constant.

**Worst Case** The needle is not in the haystack. The running time is linear in the size of the haystack.

**Average Case** The analysis depends on the distribution of the elements in the haystack. If we assume that all these are drawn uniformly at random from  $[1, N]$  for some

$N$ , then the search will terminate at the first element with probability  $1/N$ , at the second element with probability  $(1 - 1/N)1/N$  and so on. Let the random Variable  $X$  denote the number of elements. Then:

$$P(X=i) = (1-t)^{i-1}t$$

where  $t=1/N$  and:

$$E[X] = t \sum_{i=1}^n i(1-t)^{i-1}$$

which can be analyzed using the formula for arithmetic geometric series for various values of  $N$ .

Linear search only assumes that elements can be compared for equality. If we impose the stronger requirement on inputs that elements of the type are ordered, then we can use a much more efficient algorithm called binary search. We also need the elements in `haystack` to be arranged in that order.

```

1 template <typename T>
2 T* search (
3     T* haystack, size_t n,
4     const T& needle
5 )
6 {
7     size_t begin = 0, end = n;
8
9     while (begin < end) {
10        size_t mid = begin + (end - begin) / 2;
11        if (haystack[mid] < needle)
12            begin = mid + 1;
13        else if (haystack[mid] > needle)
14            end = mid;
15        else
16            return &haystack[mid];
17    }
18    return nullptr;
19 }

```

Here's how the search works on an example array while looking for a 6. The `~` marker indicates the element compared with the needle. The size of the sub-array to be searched thereafter is roughly halved after each comparison. We show the values of `begin` and `end` in the last two columns.

```

1 2 4 5 9 10 | 0 6
      ~
      9 10 | 4 6
            ~
            9 | 4 5
                  ~
                  | 4 4

```

**Ex. 23** — Analyze the average case running time for linear search in the following cases:

1.  $N=10$ .

2.  $N=\sqrt{n}$ .

3.  $N=n$ .

4.  $N=n^2$ .

5.  $N=2^n$ .

Check the results of your analysis by measuring running times.

**Ex. 24** — Analyze the running time of Binary search in the best, worst, and average cases.

**Ex. 25** — Let  $i$  be the index of a needle in the haystack or  $N$  if it is absent from the haystack. Design an algorithm that runs in time proportional to  $\log(i)$  to search for the needle in the haystack if the elements are arranged in order.

**Ex. 26** — Given an  $n$ -element array  $A$  of integers that is obtained by rotating an array sorted in increasing order and an element  $v$  as input, output the index of  $v$  in  $A$  if it exists or  $-1$ . To illustrate the property of the input array, note that the input array  $\{4, 5, 6, 1, 2, 3\}$  is obtained by rotating  $\{1, 2, 3, 4, 5, 6\}$  three places to the right.

**Ex. 27** — Given two arrays  $A$  and  $B$  of  $n$  and  $m$  integers respectively such that both  $A$  and  $B$  are sorted in increasing order, find the median of these  $m+n$  elements.

**Ex. 28** — Given an  $n \times n$  array  $A$  of integers, where each row and column is sorted in increasing order (all numbers are distinct), and a natural number  $k \in [0, n)$  as input, output the  $k^{\text{th}}$  smallest number in the matrix.

**Ex. 29** — Write a program to calculate the minimum truck capacity required to ship some packages within  $D$  days. The input is an  $n$ -element array  $A$  of package weights and the number  $D$ . The packages have to be shipped in order. That is, a package  $A[i]$  cannot be shipped on a later day as  $A[j]$  if  $i < j$ . If the capacity is  $W$ , then packages weighing up to  $W$  can be shipped in a day.

## 6 Asymptotic Notation

Asymptotic notation is a machine independent way to denote the performance of algorithms. We define the following set parameterized by functions on natural numbers:

$O(f) = \{g \mid \text{for some } c \text{ and } n_0, g(n) \leq cf(n) \text{ for } n \geq n_0\}$   
 where  $f, g: \mathbb{N} \mapsto \mathbb{N}$ ,  $c \in \mathbb{Q}$ , and  $n_0 \in \mathbb{N}$ .

Let us analyze binary search in the worst case. Let  $k$  be the maximum amount of time taken by a single iteration of the main loop. Then, the running time  $T(n)$  spent by the main loop on a haystack of size  $n$  is:

$$T(n) \leq T(\lceil n/2 \rceil) + k$$

when  $n > 1$  and  $T(0) \leq T(1) \leq k$ .

Observing  $T(n)$  for different values of  $n$ , we see:

| $n$      | $T(n)$   |
|----------|----------|
| 1        | $k$      |
| 2        | $2k$     |
| 3        | $3k$     |
| 4        | $3k$     |
| $\vdots$ | $\vdots$ |
| 8        | $4k$     |
| $\vdots$ | $\vdots$ |
| 16       | $5k$     |

The interesting cases are when  $n$  is a power of two. Observe  $T(n)$  is  $k, 2k, 3k, \dots$ , for  $n$  in  $1, 2, 4, \dots$ . For the values of  $n$  not a power of two, the value  $T(n)$  is bounded by its value at the next power of two. We can formalize this using a proof by induction and obtain  $T \in O(\log)$ . This is usually written as  $T(n) = O(\log n)$ .

*Proof.* We split the proof into two cases and show that in both cases  $T(n) = O(\log n)$ .

We first consider the case when  $n$  is a power of two. Say  $n = 2^p$ , then we claim  $T(n) \leq k(p+1)$ . The base case has  $p=0$  and we know  $T(1) \leq k$ . For the inductive case, we have  $T(2^p) \leq T(2^{p-1}) + k \leq kp + k = k(p+1)$ .

When  $n$  is not a power of two, we have a  $p$  such that  $2^p \leq n < 2^{p+1}$ . The running time can be bounded as  $T(n) \leq T(2^{p+1}) \leq k(p+2) \leq (k+1)\log n$  for  $n \geq 2^{2k}$ .  $\square$

We also need to show that in the worst case, binary search needs at least  $\log n$  time. This cannot be done using  $O(\cdot)$  notation as it only provides an upper bound. We define another parameterized set:

$$\Omega(f) = \{g \mid \text{for some } c \text{ and } n_0, f(n) \geq cg(n) \text{ for } n \geq n_0\}$$

A worst case input for binary search is when the needle is absent from the haystack. Let  $k$  be the minimum amount of time taken by one iteration of the loop. We observe

$T(1) \geq k, T(2) \geq 2k, \dots$ . As before, we can use induction to prove  $T(n) = \Omega(\log n)$ .

We also have the set  $\Theta(f) = O(f) \cap \Omega(f)$  that provides a single notation for both upper and lower bounds for a function on natural numbers.

**Ex. 30** — What is the best case running time for binary search in asymptotic notation?

**Ex. 31** — Show that the average case running time for linear search is  $T(n) = \Theta(n)$ .

**Ex. 32** — Suppose  $f(n) \leq kg(n) + c$  for some constants  $k, c$ , and  $n_0$  for all  $n \geq n_0$ . Prove that  $f = O(g)$ .

**Ex. 33** — Prove: If  $f = O(g)$  and  $g = O(h)$ , then  $f = O(h)$ .

**Ex. 34** — Prove: If  $f = g_1 + g_2$  and  $g_1, g_2 \in O(h)$ , then  $f = O(h)$ .

**Ex. 35** — Prove: If  $f = g_1 g_2$ ,  $g_1 \in O(h_1)$ , and  $g_2 \in O(h_2)$ , then  $f = O(h_1 h_2)$ .

## 7 Sorting

The sorting problem is: given an array  $A$  of  $n$  elements of some type with an ordering relation  $\leq$ , rearrange the elements so that  $A[0] \leq A[1] \leq \dots \leq A[n-1]$ .

The first algorithm we study for sorting is insertion sort. The code is:

```
1 template <typename T>
2 void sort(T* xs, size_t n)
3 {
4     if (n == 0)
5         return;
6
7     for (size_t i = 1; i < n; ++i) {
8         T key = xs[i];
9         ssize_t j = i - 1;
10        while (j >= 0 && xs[j] > key) {
11            xs[j + 1] = xs[j];
12            --j;
13        }
14        xs[j + 1] = key;
15    }
16 }
```

where lines 8 to 14 implement insertion. Observe that since  $j$  could become negative, we use `ssize_t`, a signed integer type.

The core component of insertion sort is a procedure called insertion. We have the precondition that we are given a sorted array  $A[0 \dots i-1]$  of  $i$  elements and an element at  $A[i]$ , say  $v$ . The goal or required postcondition is to make  $A[0 \dots i]$  sorted. We know that there must be an index  $j$  such that  $A[j] \leq v$  for  $k \leq j$  and  $A[k] > v$  for  $k > j$ . So we can achieve our goal by shifting  $A[j+1 \dots i-1]$  one place to the right and setting  $A[j+1]$  to  $v$ . Observe that when  $i=1$ , the precondition is met and we can apply the insertion procedure successively to satisfy the precondition for  $i=2$  to  $n-1$ . Notice that the postcondition when  $i=n-1$  is that the whole array is sorted.

The asymptotic analysis for the running time of insertion sort is as follows:

**Best Case** The input is a sorted array of  $n$  elements. The insertion procedure takes  $\Theta(1)$  time for all  $i$ . Therefore, the total time is  $\Theta(1) \times n = \Theta(n)$ .

**Worst Case** The input is the reverse of a sorted array. The insertion procedure takes  $\Theta(i)$  for all  $i$ . Therefore, the total time is  $\sum_{i=1}^n \Theta(i) = \Theta(n^2)$ .

**Average Case** Assume that the input is a permutation of  $[1, n]$  distributed uniformly at random. Let  $X_i$  be the

random variable that is the number of elements to be shifted in the  $i^{\text{th}}$  iteration. With probability  $1/2$ , the element at position  $i+1$  in the input is in the lower-half of elements  $A[0 \dots i+1]$ . Therefore, we have  $E[X_i] = i/2$ . If  $X$  is the total number of shifts performed during the algorithm, then  $E[X] = \sum_{i=0}^{n-1} E[X_i] = \Theta(n^2)$ .

The above analysis shows that insertion sort works well when we know the input is almost sorted and that the average case is close to the worst case.

Insertion sort is very efficient in its use of space. In addition to the space required for the input array, it only uses  $O(1)$  units of storage.

The second sorting algorithm we study for sorting is quicksort. It is a recursive algorithm based on a class of algorithms known as partition. A partition algorithm takes as input an array  $A[0 \dots n-1]$  and an element in the array called the pivot. Say the pivot is  $v$ . The postcondition of partition is to reorder the elements in the array and find a  $j$  such that  $A[0 \dots j-1]$  contains only elements less than  $v$ ,  $A[j] = v$ , and  $A[j+1 \dots n-1]$  contains only elements that are at least  $v$ .

There are multiple algorithms that solve the partition problem. The following is Lomuto partition, a simple to implement algorithm:

```

1
2 template <typename T>
3 size_t lomuto_partition (
4     T* xs, size_t low, size_t high, size_t pi
5 )
6 {
7     std::swap(xs[pi], xs[high]);
8     T pivot = xs[high];
9     size_t i = low;
10    for (size_t j = low; j < high; ++j) {
11        if (xs[j] < pivot) {
12            std::swap(xs[i], xs[j]);
13            ++i;
14        }
15    }
16    std::swap(xs[i], xs[high]);
17    return i;
18 }
```

The argument `pi` is the index of the pivot element. The array to be partitioned is `xs[low ... high]`. First, we swap the pivot and the last element. Then, we move all elements smaller than the pivot to the prefix of this array. The variable `i` is such that `xs[low ... i-1]` always contains elements less than `pivot`. Initially, we have `i == low`, or in other words, this prefix is empty. Then, we scan the array from left to right in the loop from line 10 to 15 moving any element less than the pivot to the prefix. Finally, in line 16, we swap the pivot element, which was initially moved to the last position in the array with `xs[i]`. The return value of the algorithm is the index of the pivot element after the partition. Observe that the running time of Lomuto partition algorithm is  $\Theta(n)$ .

The quicksort algorithm is:

```

1 template <typename T>
2 void quicksort(T* xs, size_t low, size_t high)
3 {
4     if (low < high) {
5         size_t mid = low + (high - low) / 2;
6         size_t pi = mo3(xs, low, mid, high);
7         size_t p = lomuto_partition (
8             xs, low, high, pi
9         );
10        if (p > 0)
11            quicksort(xs, low, p - 1);
12        quicksort(xs, p + 1, high);
13    }
14 }
15
16 template <typename T>
```

```

17 void sort(T* xs, size_t n)
18 {
19     if (n > 0)
20         quicksort(xs, 0, n - 1);
21 }
```

The pivot index could be any valid index into the array. But, we pick the index of the median of first, middle, and last elements. This choice is experimentally known to work well. After partition, we recursively sort the left and right sub-arrays in lines 11 and 12. Notice that each of these arrays are strictly smaller than the input array as we exclude the pivot element from both of them. Therefore, we can argue using induction that this algorithm correctly sorts the array.

The asymptotic analysis for the running time of quicksort is as follows. If we use  $T(n)$  denote the running time on the given  $n$  element array, then we can write  $T(n) = T(i) + T(n-i-1) + \Theta(n)$  for some  $0 \leq i < n$ . The cases are:

**Best Case** The pivot elements are such that  $i = n/2$ <sup>1</sup>.

Then, the recurrence is  $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ .

**Worst Case** The pivot element is the smallest element or the largest element at each step. Then, the recurrence is  $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$ .

**Average Case** Assume elements are  $[1, n]$  and all permutations are equally likely. Then, the pivot element's correct index is  $i$  with probability  $1/n$  for all  $i$ . The expected running time is:  $T(n) = \Theta(n) + \frac{1}{n} \sum_{i=0}^{n-1} T(i) + T(n-i-1)$ . Solving this, we obtain  $T(n) = \Theta(n \log n)$ . With a more detailed analysis, it can be shown that the average case for quicksort is only 39% worse than its best case.

The median of three pivot selection works well if the input arrays are close to random permutations. But, for any deterministic method for picking the pivot such as picking any fixed position or picking the median of three elements, we can construct inputs that degrade the running time of quicksort to  $\Theta(n^2)$ . Therefore, if the sorting algorithm is running on a web server, a clever attacker can feed carefully chosen arrays to degrade the performance of the server. If we expect the sorting algorithm to receive such untrusted inputs, then we can avoid this performance degradation is to choose the pivot element randomly. However, choosing random pivots consumes random bits supplied by the OS, a scarce resource. So if we want to avoid this worst case, we have to find the median of the array in linear time to obtain good pivots, or use a sorting algorithm such as mergesort or heapsort that has worst case running time of  $\Theta(n \log n)$ .

**Ex. 36** — Measure the running time of insertion sort and quicksort in the best, worst, and average cases by counting the number of comparisons of array elements performed during the algorithm. Measure for input sizes from 100 to  $10^9$  multiplying the size by 10 at each step.

**Ex. 37** — Compare the performance of quicksort with the two pivot selection strategies, choosing the first element and the median of three, on randomly generated arrays.

**Ex. 38** — Implement Hoare partition. Compare its performance with Lomuto partition on randomly generated arrays.

**See Also:**

- Section 2.1 in [CLRS22] for insertion sort.
- Chapter 7 in [CLRS22] for quicksort.

<sup>1</sup>We ignore floors and ceilings

## 8 Graphs

A graph is an abstract mathematical object that can model many real-world problems. Mathematically, a graph is a set  $V$  along with a relation  $E \subseteq V \times V$ . If the set  $V$  is finite, the graph is called a *finite graph*. An element  $v \in V$  is called a *vertex* or a *node* and an element  $e = (u, v) \in E$  is called an *edge* or an *arc* with the vertex  $u$  as its *tail* and  $v$  as its *head*. We sometimes say  $e$  is an edge from  $u$  to  $v$  to specify the direction. An edge  $(u, u)$  is called a *self-loop*. A graph without self-loops is called *simple*. A graph is called *undirected* if for all  $u, v \in V$ , we have  $(u, v) \in E$  if and only if  $(v, u) \in E$ . We say that  $u$  and  $v$  are *adjacent* if  $(u, v) \in E$  in an undirected graph. For a vertex  $v$ , we use  $d_+(v)$  to denote the number of edges where  $v$  is the tail and  $d_-(v)$  is the number of edges where  $v$  is the head.

A graph has many interesting sub-objects. We define some of them here. A *walk* from  $u$  to  $v$  is a sequence  $(v_0 = u, v_1, \dots, v_k, v_{k+1} = v)$  such that  $(v_i, v_{i+1})$  is an edge for  $0 \leq i \leq k$ . The *length of the walk* is the number of edges in it ( $k+1$  above). If  $u$  and  $v$  are distinct, the walk is *open*, otherwise it is *closed*. An open walk is called a *path* if all  $v_i$  are pairwise distinct. A closed walk is called a *cycle* if all  $v_i$  are pairwise distinct except  $v_0$  and  $v_{k+1}$ .

Many real-world problems can be restated as problems on graphs. Consider the undirected graph where  $V$  corresponds to the cells on a chessboard and  $(u, v)$  is an edge if and only if  $u$  and  $v$  are in the same row, column, or diagonal. The problem of placing  $n$  non-threatening queens on the chessboard is then the same as finding a subset  $U \subseteq V$  such that  $|U| = n$  and there are no edges between vertices in  $U$ . Such a subset is called an *independent set* of the graph.

Consider the undirected graph where  $V$  corresponds to the cells in a SUDOKU puzzle. Two vertices are adjacent if and only if they are in the same row, column, or box. Some vertices are assigned a color which is a number from 1 to 9 such that adjacent vertices do not have the same color. The SUDOKU puzzle is then to extend this coloring to all vertices while preserving the property that adjacent vertices do not have the same color. The general version of this problem is called the *graph coloring* problem.

## 9 Graph Representations

There are three popular ways to represent graphs on a computer. Two of those, the adjacency matrix and adjacency list representations, are explicit. The third one is an implicit representation based on functions which is useful when the graph is too large to fit into memory.

For the explicit representations, we assume that the vertex set is  $[0, n)$ . The adjacency matrix representation is a matrix  $A[0 \dots n-1][0 \dots n-1]$  such that  $A[i][j] = 1$  if there is an edge from  $i$  to  $j$  and 0 otherwise. The adjacency list representation is an array  $L[0 \dots n-1]$  where  $L[i]$  is a linked list of vertices  $j$  such that  $(i, j)$  is an edge in the graph.

Let  $m$  be the number of edges in the graph. We compare the two explicit representations by considering the time complexity for the following operations that are common to many graph algorithms:

1. Given  $(i, j)$ , is it an edge in the graph?
2. Given  $i$ , iterating over all  $j$  such that  $(i, j)$  is an edge in the graph.
3. Iterating over all edges in the graph.

The comparison for these two operations are given in Table 1. where  $d_+(v) < n$  for any vertex  $v$  and  $m \leq n^2$ .

## 10 Depth-first Search

Depth-first search (DFS) is a graph traversal algorithm. A graph traversal algorithm is a way to systematically visit all

| Op | Adjacency Matrix | Adjacency List   |
|----|------------------|------------------|
| 1  | $O(1)$           | $\Theta(n)$      |
| 2  | $\Theta(n)$      | $\Theta(d_+(v))$ |
| 3  | $\Theta(n^2)$    | $\Theta(n+m)$    |

Table 1: Comparison of adjacency matrix and adjacency list representations

the vertices and edges of the graph. As its name suggests, a depth-first search moves from the current vertex to an unvisited child at each step, backtracking as necessary to prevent exploring the same vertex again.

Consider the graph given by the following adjacency list:

```
0: 1 2
1: 3
2: 4 5
3:
4:
5: 0 4
```

DFS traverses the vertices of the graph in the following order:

```
0
-> 1
-> 3
<- 1
<- 0
-> 2
-> 4
<- 2
-> 5
(0 and 4 are already visited)
<- 2
<- 0
```

The traversal yields two functions from vertices to natural numbers – discovery time  $d[v]$  and finishing time  $f[v]$ . They are the time at which a vertex is discovered and the time at which we have finished exploring the vertex and everything from it. It also computes a forest (a collection of trees) as a function  $p$  from vertices to vertices or  $-1$ . The value  $p[v]$  is either the parent of  $v$  in the forest or  $-1$  if  $v$  is a root. For the above example, we have:

| v | d | f  | p  |
|---|---|----|----|
| 0 | 1 | 12 | -1 |
| 1 | 2 | 5  | 0  |
| 2 | 6 | 11 | 0  |
| 3 | 3 | 4  | 1  |
| 4 | 7 | 8  | 2  |
| 5 | 9 | 10 | 2  |

The main part of a DFS is the following function to visit all vertices of a graph starting from a given start vertex:

```
1 void do_dfs (
2   struct graph *g, size_t u,
3   bool *visited,
4   size_t *d, size_t *f, ssize_t *p,
5   size_t *timer
6 ) {
7   visited[u] = true;
8   d[u] = ++(*timer);
9
10  for (
11    struct node *node = g->adj[u];
12    node;
13    node = node->next
14  ) {
15    size_t v = node->to;
16    if (!visited[v]) {
17      p[v] = u;
18      do_dfs(g, v, visited, d, f, p, timer);
19    }
20  }
21
22  f[u] = ++(*timer);
23 }
```

```

24
25 void dfs_visit (
26     struct graph *g,
27     size_t start,
28     bool *visited,
29     size_t *d, size_t *f, ssize_t *p,
30     size_t *timer
31 ) {
32     p[start] = -1;
33     do_dfs(g, start, visited, d, f, p, timer);
34 }

```

The array value `visited[v]` is `true` if and only if  $v$  is already visited. The vertex `start` is a root of the DFS forest. We start the visit at this vertex in line 33. The function `do_dfs()` is the key procedure. In the for loop from line 10 to 22, we iterate over all the unvisited out neighbors of the current vertex  $u$  and recursively DFS from it.

The main DFS procedure simply calls `dfs_visit()` one-by-one from each unvisited vertex.

```

1 void dfs (
2     struct graph *g,
3     size_t *d, size_t *f, ssize_t *p
4 ) {
5     bool *visited = calloc(g->n, sizeof(bool));
6     size_t timer = 0;
7
8     for (size_t i = 0; i < g->n; i++) {
9         if (!visited[i]) {
10             dfs_visit (
11                 g, i, visited,
12                 d, f, p,
13                 &timer
14             );
15         }
16     }
17
18     free(visited);
19 }

```

Observe that the for loop in `do_dfs()` over all the calls to that function looks at each edge exactly once. Moreover, `dfs_visit()` is called for each vertex exactly once. So the time complexity is  $\Theta(n+m)$ . We store `visited` array using  $\Theta(n)$  space. Observe that the functions only use a constant number of fixed size variables in their bodies. The depth of the recursion in `do_dfs()` is bounded by the length of a simple path in the graph which is  $O(n)$ . So the space complexity is  $\Theta(n)$ .

**Ex. 39** — Write depth-first search over the adjacency matrix representation for graphs.

**Ex. 40** — A *lex-first* DFS is a DFS that moves to the lowest numbered unvisited vertex at each step. Write a program to print discovery, finish, and parent arrays for lex-first DFS. What is the time complexity for (a) adjacency list (b) adjacency matrix?

### 10.1 Minimax Algorithm

The minimax algorithm can be used to compute optimal strategies for turn-based, two-player games. We illustrate this application for DFS by computing the optimal strategy for Tic-tac-toe.

We consider the graph where vertices are Tic-tac-toe board positions. The pair  $(u, v)$  is an edge if and only if there is a valid move that takes the game from  $u$  to  $v$ . Our goal is to compute a best move from each vertex in this graph.

We use the following idea to compute the optimal move and the optimal result for the player (say A) moving at  $u$  assuming that the other player will play optimally: consider all  $v$  such that  $(u, v)$  is an edge and compute the optimal move and the optimal result for the other player (say B) from  $v$  recursively. If the result with optimal play is a loss

for B from  $v$ , then moving to  $v$  is a best move for A. If the result with optimal play is a win for B from all such  $v$ , then A has no winning moves from  $u$ . Otherwise, there is a  $v$  such that the optimal play for B leads to a draw and therefore the optimal play for A is to move to  $v$ .

First, we have to implement some basic functions to gather information about the board and manipulate it. We represent the board using a `uint32_t` as follows: each cell is assigned a place value in base-3 representation. A blank cell is the digit 0, an X is 1, and an O is 2. The largest value possible in this encoding is  $3^{16} - 1$ . The optimal move from a board is encoded using a `uint8_t` as follows: the byte is `0wdlrrcc` where `wdl` indicates whether the optimal move leads to a win, draw, or loss and `rrcc` encodes the row and column of the optimal move. So  $3^{16} - 1$  or under 45 megabytes are sufficient to store the entire strategy. The following function computes an optimal move for all boards reachable from `board`, stores it into the array `best`, and returns that optimal move.

```

1 uint8_t ttt(uint32_t board, uint8_t *best) {
2     assert(!is_done(board));
3
4     if (best[board])
5         return best[board];
6
7     int p = player(board);
8     uint8_t b = 0;
9
10    for (size_t r = 0; r < BOARD_SIZE; ++r)
11        for (size_t c = 0; c < BOARD_SIZE; ++c)
12            if (!mark(board, r, c)) {
13                uint32_t new_board = place (
14                    board, r, c, p
15                );
16                if (has_won(new_board, p))
17                    return best[board] = move(WIN, r, c);
18                if (is_done(new_board))
19                    return best[board] = move (
20                        DRAW, r, c
21                    );
22                uint8_t f = ttt(new_board, best);
23                if (is_loss(f))
24                    return best[board] = move(WIN, r, c);
25                if (is_draw(f))
26                    b = move(DRAW, r, c);
27                else if (!b)
28                    b = move(LOSS, r, c);
29            }
30    assert(b);
31    return best[board] = b;
32 }

```

In a finished game, there is no optimal move, line 2 ensures that this error is not made at the caller. If a best move for this board has already been computed, we return it in lines 4 and 5. This ensures that we don't recompute it for the same board. The variable `p` is set to the player making the move on this board. It is either 1 or 2. After this we iterate over all the out neighbors of this board in the graph. That is, we iterate over all empty cells and place this player's mark to get the out neighbor in lines 13 to 15. Lines 16 to 21 check some easy cases – the player has won by this placement or the board is full. In this case, we know an optimal move without any further computation. Otherwise, we compute an optimal move for the other player from the new board. If it is a loss, then this move is a winning one and we immediately return. If it is a draw, we store it in `b` and continue searching for a potential win or draw. Otherwise, if no move has been set, we set this move as a losing move.

### 10.2 Path and Cycle Finding

We can use DFS to find paths in graphs as follows: Say we are given a graph  $G$  and two vertices  $u$  and  $v$  in  $G$ . We have to find a path from  $u$  to  $v$  if it exists or report that no such

path exists. The algorithm is: Start a DFS visit from  $u$ . If we discover  $v$ , then the sequence of vertices  $v, p[v], p[p[v]], \dots$ , is the reverse of a path from  $u$  to  $v$ . If we do not discover  $v$  before finishing  $u$ , then there is no path from  $u$  to  $v$ .

To understand why this algorithm works, we need some definitions. During the course of a DFS, we can assign one of three colors white, gray, and black to each vertex in the graph. A vertex is white if it hasn't been discovered yet. It is gray if it has been discovered but not finished. It is black after it is finished. The correctness of the above algorithm is given by the following theorem called the *white path theorem*.

**Theorem 1.** *A vertex  $v$  is a descendant of a vertex  $u$  in a DFS tree if and only if at time  $d[u]$ , there is a path of vertices, all colored white, from  $u$  to  $v$ .*

To see the correctness of the path finding algorithm, observe that since we start the DFS at  $u$ , if there is a path from  $u$  to  $v$  in the graph, then it must be a white path.

Observe that the path finding algorithm runs in  $O(n+m)$  time.

A DFS also classifies the edges of the graph into the following four (disjoint) categories:

**Tree edges**  $(u,v) \in E(G)$  such that  $p[v] = u$ .

**Forward edges**  $(u,v) \in E(G)$  such that  $v$  is a descendant of  $u$  in the DFS forest and  $(u,v)$  is not a tree edge.

**Back edges**  $(u,v) \in E(G)$  such that  $u$  is a descendant of  $v$  in the DFS forest.

**Cross edges** All other edges.

A cycle finding algorithm using DFS is: Run DFS on  $G$ . If a back edge  $(u,v)$  is found, then that edge along with the path from  $v$  to  $u$  in the DFS forest forms a cycle. If no back edges are found during DFS, there are no cycles in the graph.

Why is this algorithm correct? If the algorithm finds a back edge, then there is a cycle in the graph as above. We have to prove that if the graph has a cycle, the algorithm will find some cycle (it need not be the same cycle). Suppose there is a cycle  $(u_0, \dots, u_k)$  in the graph, then the DFS will visit some vertex  $u_i$  in the cycle first. At that time, there is a white path from  $u_i$  to  $u_{i-1}$ . Now, one of the two things must happen. By Theorem 1, the algorithm will discover  $u_{i-1}$  before finishing  $u_i$  or some other back edge is found before discovering  $u_{i-1}$ . In the second case, we correctly report a cycle. In the first case, the edge  $(u_{i-1}, u_i)$  is then a back edge found by the algorithm and we correctly report this cycle.

Observe that the cycle finding algorithm runs in  $O(n+m)$  time.

### 10.3 Topological Sort

The topological sort problem is: Given a directed acyclic graph (DAG)  $G$ , arrange the vertices of  $G$  as  $u_1, \dots, u_n$  such that  $(u_i, u_j) \in E(G)$  if and only if  $i < j$ . This models many real-world problems such as task scheduling: The vertices of the DAG correspond to tasks where an edge  $(u,v)$  indicates that task  $u$  must be finished before beginning task  $v$ . A topological sort is a way to schedule tasks.

An algorithm for topological sort is: Run DFS on  $G$  and arrange the vertices in the decreasing order of their finishing times.

To prove the correctness of this algorithm, it is sufficient to prove that if  $f[u] < f[v]$ , then there is no path from  $u$  to  $v$  in  $G$ . First, we prove that during a DFS in a DAG, if a vertex  $u$  is colored black, then all its out-neighbors are also black. Clearly, an out-neighbor  $v$  cannot be white (even in graphs with cycles). If  $v$  is gray, then  $u$  is a descendant of  $v$  in the DFS forest and  $(u,v)$  is a back edge. But this is impossible since  $G$  does not have cycles. So  $v$  is black. Now,

suppose  $f[u] < f[v]$  and there is a path from  $u$  to  $v$ . At time  $f[u]$ ,  $u$  is black, therefore all nodes from  $u$  to  $v$  including  $v$  must be black at  $f[u]$ . So  $f[v] < f[u]$ .

Observe that the running-time for the topological sort algorithm is  $O(n+m)$ .

### 10.4 Strongly Connected Components

We define an equivalence relation on vertices as follows:  $u$  is related to  $v$  if there are paths from  $u$  to  $v$  and  $v$  to  $u$  in the graph. For an equivalence class  $U \subseteq V$  of this relation, the induced subgraph  $G[U]$  is a *strongly connected component* of  $G$ . The strongly connected component problem is to determine the strongly connected components of  $G$  given  $G$  as input.

An algorithm for this problem given by Kosaraju is: Run DFS( $G$ ) and compute  $f[u]$  for each  $u$ . Compute  $Rev(G)$ , the graph obtained from  $G$  by reversing the direction of every edge<sup>2</sup>. Run DFS( $Rev(G)$ ) by choosing  $u$  with largest  $f[u]$  as root at each iteration. The DFS trees produced by the second DFS span the strongly connected components of  $G$ .

To prove the correctness of the algorithm, we observe how can the algorithm be wrong?

### 11 Breadth-first Search

### 12 Shortest Paths

### 13 Minimum Spanning Trees

### 14 Hash Tables

### 15 Binary Search Trees

### 16 Balanced Binary Search Trees

### 17 Full Source Code for Examples

Brute-force Search for  $n$ -queens

```
1 #include <assert.h>
2 #include <stdbool.h>
3 #include <stdint.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7
8 void swap(size_t *a, size_t *b)
9 {
10     size_t t = *a;
11     *a = *b;
12     *b = t;
13 }
14
15 void reverse(size_t *a, size_t b, size_t e)
16 {
17     while (b < e) {
18         swap(&a[b], &a[e]);
19         ++b;
20         --e;
21     }
22 }
23
24 void first(size_t n, size_t *queens)
25 {
26     for (size_t i = 0; i < n; ++i)
27         queens[i] = i;
28 }
29
30 bool next(size_t n, size_t *queens)
31 {
32     ssize_t i = n-2;
33     while (i >= 0 && queens[i] >= queens[i+1])
34         --i;
35     if (i < 0)
36         return false;
37     size_t j = n-1;
38     while (queens[j] <= queens[i])
39         --j;
40     swap(&queens[i], &queens[j]);
```

<sup>2</sup>This graph is also called the transpose of  $G$ .

```

41 reverse(queens, i+1, n-1);
42 return true;
43 }
44
45 bool ok(size_t n, size_t *queens)
46 {
47     for (size_t i = 0; i < n; ++i)
48         for (size_t j = i+1; j < n; ++j) {
49             size_t d;
50             if (queens[i] < queens[j])
51                 d = queens[j]-queens[i];
52             else
53                 d = queens[i]-queens[j];
54             if (d == j-i)
55                 return false;
56         }
57     return true;
58 }
59
60 size_t nqueens(size_t n)
61 {
62     size_t c = 0;
63     size_t *queens = malloc(n * sizeof(size_t));
64     assert(queens);
65     first(n, queens);
66     while (true) {
67         if (ok(n, queens))
68             ++c;
69         if (!next(n, queens))
70             break;
71     }
72     return c;
73 }
74
75 int main(int argc, const char *argv[])
76 {
77     size_t n = 8;
78     if (argc > 1) {
79         n = 0;
80         const char *arg = argv[1];
81         while (*arg)
82             n = n * 10 + (*arg++ - '0');
83     }
84     printf("%ld\n", nqueens(n));
85     return 0;
86 }

```

### Backtracking Search for $n$ -queens

```

1 #include <assert.h>
2 #include <stdbool.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 struct node {
7     size_t n;
8     size_t *queens;
9     size_t k;
10    ssize_t last;
11 };
12
13 void root(size_t n, struct node *p)
14 {
15     p->n = n;
16     p->k = 0;
17     p->last = -1;
18     p->queens = malloc(n * sizeof(size_t));
19     assert(p->queens);
20 }
21
22 bool ok(const struct node *p)
23 {
24     for (size_t i = 0; i < p->k; ++i)
25         for (size_t j = i+1; j < p->k; ++j) {
26             size_t d;
27             if (p->queens[i] < p->queens[j])
28                 d = p->queens[j]-p->queens[i];
29             else
30                 d = p->queens[i]-p->queens[j];
31             if (d == j-i)
32                 return false;
33         }
34     return true;

```

```

35 }
36
37 bool up(struct node *p)
38 {
39     if (p->k == 0)
40         return false;
41     p->last = p->queens[p->k-1];
42     --(p->k);
43     return true;
44 }
45
46 bool down(struct node *p)
47 {
48     bool *used = malloc(p->n * sizeof(bool));
49     assert(used);
50     for (size_t i = 0; i < p->n; ++i)
51         used[i] = false;
52     for (size_t i = 0; i < p->k; ++i)
53         used[p->queens[i]] = true;
54
55     for (size_t i = p->last+1; i < p->n; ++i)
56         if (!used[i]) {
57             p->queens[p->k] = i;
58             ++(p->k);
59             p->last = -1;
60             free(used);
61             return true;
62         }
63     free(used);
64     return false;
65 }
66
67 bool leaf(const struct node *p)
68 {
69     return p->k == p->n;
70 }
71
72 size_t nqueens(size_t n)
73 {
74     struct node p;
75     root(n, &p);
76     size_t c = 0;
77     while (true) {
78         if (leaf(&p)) {
79             if (ok(&p))
80                 ++c;
81             if (!up(&p))
82                 break;
83         } else {
84             if (!ok(&p)) {
85                 if (!up(&p))
86                     break;
87             } else {
88                 if (!down(&p))
89                     if (!up(&p))
90                         break;
91             }
92         }
93     }
94     return c;
95 }
96
97 int main(int argc, const char *argv[])
98 {
99     size_t n = 8;
100    if (argc > 1) {
101        n = 0;
102        const char *arg = argv[1];
103        while (*arg)
104            n = n * 10 + (*arg++ - '0');
105    }
106    printf("%ld\n", nqueens(n));
107    return 0;
108 }

```

### Recursive Generation of All Permutations

```

1 #include <stdio.h>
2
3 void print(const int *a, size_t n)
4 {
5     printf("<_");
6     for (size_t i = 0; i < n; ++i)

```

```

7     printf("%d_", a[i]);
8     printf(">\n");
9 }
10
11 void swap(int *a, int *b)
12 {
13     int t = *a;
14     *a = *b;
15     *b = t;
16 }
17
18 void do_permutations (
19     int *a, size_t n,
20     int *b, size_t k,
21     size_t k1
22 )
23 {
24     if (k1 == k) {
25         print(b, k);
26         return;
27     }
28
29     for (size_t i = 0; i < n; ++i) {
30         swap(&a[0], &a[i]);
31         b[k1] = a[0];
32         do_permutations(a+1, n-1, b, k, k1+1);
33         swap(&a[0], &a[i]);
34     }
35 }
36
37 void permutations (
38     int *a, size_t n,
39     int *b, size_t k
40 )
41 {
42     do_permutations(a, n, b, k, 0);
43 }
44
45 int main()
46 {
47     int a[] = {0, 1, 2, 3};
48     int b[2] = {0};
49
50     permutations(a, 4, b, 2);
51
52     return 0;
53 }

```

### Recursive Generation of All Combinations

```

1 #include <stdio.h>
2
3 void print(const int *a, size_t n)
4 {
5     printf("<_");
6     for (size_t i = 0; i < n; ++i)
7         printf("%d_", a[i]);
8     printf(">\n");
9 }
10
11 void do_combinations (
12     const int *a, size_t n,
13     int *b, size_t k,
14     size_t k1
15 )
16 {
17     if (n < k-k1)
18         return;
19
20     if (k1 == k) {
21         print(b, k);
22         return;
23     }
24
25     b[k1] = a[0];
26     do_combinations(a+1, n-1, b, k, k1+1);
27     do_combinations(a+1, n-1, b, k, k1);
28 }
29
30 void combinations (
31     const int *a, size_t n,
32     int *b, size_t k
33 )

```

```

34 {
35     do_combinations(a, n, b, k, 0);
36 }
37
38 int main()
39 {
40     const int a[] = {0, 1, 2, 3};
41     int b[2] = {0};
42
43     combinations(a, 4, b, 2);
44
45     return 0;
46 }

```

### Recursive Backtracking Search for *n*-queens

```

1 #include <assert.h>
2 #include <stdbool.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 bool ok(const size_t *queens, size_t k)
7 {
8     for (size_t i = 0; i < k; ++i)
9         for (size_t j = i+1; j < k; ++j) {
10             size_t d;
11             if (queens[i] < queens[j])
12                 d = queens[j] - queens[i];
13             else
14                 d = queens[i] - queens[j];
15             if (d == j-i)
16                 return false;
17         }
18     return true;
19 }
20
21 size_t do_nqueens (
22     size_t n, size_t *queens, size_t k
23 )
24 {
25     if (k == n && ok(queens, k))
26         return 1;
27
28     if (!ok(queens, k))
29         return 0;
30
31     bool *used = malloc(n * sizeof(bool));
32     assert(used);
33     for (size_t i = 0; i < n; ++i)
34         used[i] = false;
35     for (size_t i = 0; i < k; ++i)
36         used[queens[i]] = true;
37
38     size_t c = 0;
39     for (size_t i = 0; i < n; ++i)
40         if (!used[i]) {
41             queens[k] = i;
42             c += do_nqueens(n, queens, k+1);
43         }
44     free(used);
45     return c;
46 }
47
48 size_t nqueens(size_t n)
49 {
50     size_t *queens = malloc(n * sizeof(size_t));
51     assert(queens);
52     size_t c = do_nqueens(n, queens, 0);
53     free(queens);
54     return c;
55 }
56
57 int main(int argc, const char *argv[])
58 {
59     size_t n = 8;
60     if (argc > 1) {
61         n = 0;
62         const char *arg = argv[1];
63         while (*arg)
64             n = n * 10 + (*arg++ - '0');
65     }
66     printf("%ld\n", nqueens(n));
67     return 0;

```

68 }

### Generic Linear Search

```
1 #include <cassert>
2 #include <iostream>
3
4 template <typename T>
5 T* search (
6     T* haystack, size_t n,
7     const T& needle
8 )
9 {
10     for (size_t i = 0; i < n; ++i)
11         if (haystack[i] == needle)
12             return &haystack[i];
13     return nullptr;
14 }
15
16 int main()
17 {
18     int haystack[] = { 5, 9, 1, 4, 2, 10 };
19     assert(search(haystack, 6, 4));
20     assert(!search(haystack, 6, 11));
21     return 0;
22 }
```

### Binary Search

```
1 #include <cassert>
2 #include <iostream>
3
4 template <typename T>
5 T *search (
6     T *haystack, size_t n,
7     const T& needle
8 )
9 {
10     size_t begin = 0, end = n;
11
12     while (begin < end) {
13         size_t mid = begin + (end - begin) / 2;
14         if (haystack[mid] < needle)
15             begin = mid + 1;
16         else if (haystack[mid] > needle)
17             end = mid;
18         else
19             return &haystack[mid];
20     }
21     return nullptr;
22 }
23
24 int main()
25 {
26     int haystack[] = { 1, 2, 4, 5, 9, 10 };
27     assert(search(haystack, 6, 4));
28     assert(!search(haystack, 6, 11));
29     return 0;
30 }
```

### Insertion Sort

```
1 #include <cassert>
2
3 template <typename T>
4 void sort(T* xs, size_t n)
5 {
6     if (n == 0)
7         return;
8
9     for (size_t i = 1; i < n; ++i) {
10         T key = xs[i];
11         ssize_t j = i - 1;
12         while (j >= 0 && xs[j] > key) {
13             xs[j + 1] = xs[j];
14             --j;
15         }
16         xs[j + 1] = key;
17     }
18 }
19
20 template <typename T>
```

```
21 bool is_sorted(T* xs, size_t n)
22 {
23     if (n == 0)
24         return true;
25
26     for (size_t i = 0; i < n - 1; ++i)
27         if (xs[i] > xs[i+1])
28             return false;
29     return true;
30 }
31
32 int main()
33 {
34     int xs[] = { 5, 4, 9, 3, 2, 6 };
35     sort(xs, 6);
36     assert(is_sorted(xs, 6));
37     return 0;
38 }
```

### Quicksort

```
1 #include <cassert>
2 #include <iostream>
3
4 template <typename T>
5 size_t mo3 (
6     T* xs, size_t a, size_t b, size_t c
7 )
8 {
9     if ((xs[a] < xs[b]) != (xs[a] < xs[c]))
10         return a;
11     else if ((xs[b] < xs[a]) != (xs[b] < xs[c]))
12         return b;
13     else
14         return c;
15 }
16
17 template <typename T>
18 size_t lomuto_partition (
19     T* xs, size_t low, size_t high, size_t pi
20 )
21 {
22     std::swap(xs[pi], xs[high]);
23     T pivot = xs[high];
24     size_t i = low;
25     for (size_t j = low; j < high; ++j) {
26         if (xs[j] < pivot) {
27             std::swap(xs[i], xs[j]);
28             ++i;
29         }
30     }
31     std::swap(xs[i], xs[high]);
32     return i;
33 }
34
35 template <typename T>
36 void quicksort(T* xs, size_t low, size_t high)
37 {
38     if (low < high) {
39         size_t mid = low + (high - low) / 2;
40         size_t pi = mo3(xs, low, mid, high);
41         size_t p = lomuto_partition (
42             xs, low, high, pi
43         );
44         if (p > 0)
45             quicksort(xs, low, p - 1);
46         quicksort(xs, p + 1, high);
47     }
48 }
49
50 template <typename T>
51 void sort(T* xs, size_t n)
52 {
53     if (n > 0)
54         quicksort(xs, 0, n - 1);
55 }
56
57 template <typename T>
58 bool is_sorted(T* xs, size_t n)
59 {
60     if (n == 0)
61         return true;
62 }
```

```

63 for (size_t i = 0; i < n - 1; ++i)
64     if (xs[i] > xs[i+1])
65         return false;
66 return true;
67 }
68
69 int main()
70 {
71     int xs[] = { 5, 4, 9, 3, 2, 6 };
72     sort(xs, 6);
73     assert(is_sorted(xs, 6));
74     return 0;
75 }

```

### Depth-first Search

```

1 #include <stdbool.h>
2 #include <stddef.h>
3 #include <stdint.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6
7 struct node {
8     size_t to;
9     struct node *next;
10 };
11
12 struct graph {
13     struct node **adj;
14     size_t n;
15 };
16
17 struct graph*new_graph(size_t n) {
18     struct graph *g = malloc (
19         sizeof(struct graph)
20     );
21     g->n = n;
22     g->adj = calloc(n, sizeof(struct node *));
23     return g;
24 }
25
26 void add_edge (
27     struct graph *g, size_t from, size_t to
28 ) {
29     struct node *new_node = malloc (
30         sizeof(struct node)
31     );
32     new_node->to = to;
33     new_node->next = g->adj[from];
34     g->adj[from] = new_node;
35 }
36
37 void do_dfs (
38     struct graph *g, size_t u,
39     bool *visited,
40     size_t *d, size_t *f, ssize_t *p,
41     size_t *timer
42 ) {
43     visited[u] = true;
44     d[u] = ++(*timer);
45
46     for (
47         struct node *node = g->adj[u];
48         node;
49         node = node->next
50     ) {
51         size_t v = node->to;
52         if (!visited[v]) {
53             p[v] = u;
54             do_dfs(g, v, visited, d, f, p, timer);
55         }
56     }
57     f[u] = ++(*timer);
58 }
59
60
61 void dfs_visit (
62     struct graph *g, size_t start,
63     bool *visited,
64     size_t *d, size_t *f, ssize_t *p,
65     size_t *timer
66 ) {
67     p[start] = -1;

```

```

68     do_dfs(g, start, visited, d, f, p, timer);
69 }
70
71 void dfs (
72     struct graph *g,
73     size_t *d, size_t *f, ssize_t *p
74 ) {
75     bool *visited = calloc(g->n, sizeof(bool));
76     size_t timer = 0;
77
78     for (size_t i = 0; i < g->n; i++) {
79         if (!visited[i]) {
80             dfs_visit (
81                 g, i,
82                 visited,
83                 d, f, p,
84                 &timer
85             );
86         }
87     }
88
89     free(visited);
90 }
91
92 void free_graph(struct graph *g) {
93     for (size_t i = 0; i < g->n; i++) {
94         struct node *node = g->adj[i];
95         while (node) {
96             struct node *tmp = node;
97             node = node->next;
98             free(tmp);
99         }
100     }
101     free(g->adj);
102     free(g);
103 }
104
105 int main() {
106     struct graph *g = new_graph(6);
107
108     add_edge(g, 0, 2);
109     add_edge(g, 0, 1);
110     add_edge(g, 1, 3);
111     add_edge(g, 2, 5);
112     add_edge(g, 2, 4);
113     add_edge(g, 5, 4);
114     add_edge(g, 5, 0);
115
116     size_t *d = malloc(g->n * sizeof(size_t));
117     size_t *f = malloc(g->n * sizeof(size_t));
118     ssize_t *p = malloc(g->n * sizeof(ssize_t));
119
120     for (size_t i = 0; i < g->n; i++) {
121         d[i] = f[i] = SIZE_MAX;
122         p[i] = -1;
123     }
124
125     dfs(g, d, f, p);
126
127     printf("%4s%4s%4s%4s\n", "v", "d", "f", "p");
128     for (size_t i = 0; i < g->n; i++) {
129         printf (
130             "%4zu%4zu%4zu%4zd\n",
131             i,
132             d[i], f[i], p[i]
133         );
134     }
135
136     free(d);
137     free(f);
138     free(p);
139     free_graph(g);
140
141     return 0;
142 }

```

### 4x4 Tic-tac-toe

```

1 #include <assert.h>
2 #include <stdbool.h>
3 #include <stddef.h>
4 #include <stdint.h>
5 #include <stdio.h>

```

```

6 #include <stdlib.h>
7
8 #define BOARD_SIZE (4)
9
10 enum { LOSS = 1, DRAW = 2, WIN = 4 };
11
12 size_t power(size_t a, size_t n) {
13     size_t p = 1;
14     while (n-- > 0)
15         p *= a;
16     return p;
17 }
18
19 int mark(uint32_t board, size_t r, size_t c) {
20     size_t n = BOARD_SIZE * r + c;
21     return (board / power(3, n)) % 3;
22 }
23
24 uint32_t place (
25     uint32_t board,
26     size_t r, size_t c, int p
27 ) {
28     size_t n = BOARD_SIZE * r + c;
29     return board + p * power(3, n);
30 }
31
32 bool is_full(uint32_t board) {
33     for (size_t r = 0; r < BOARD_SIZE; ++r)
34         for (size_t c = 0; c < BOARD_SIZE; ++c)
35             if (!mark(board, r, c))
36                 return false;
37     return true;
38 }
39
40 bool has_won(uint32_t board, int m) {
41     for (size_t r = 0; r < BOARD_SIZE; ++r) {
42         for (size_t c = 0; c < BOARD_SIZE; ++c) {
43             if (mark(board, r, c) != m)
44                 goto next_row;
45         }
46         return true;
47     next_row:;
48     }
49     for (size_t c = 0; c < BOARD_SIZE; ++c) {
50         for (size_t r = 0; r < BOARD_SIZE; ++r) {
51             if (mark(board, r, c) != m)
52                 goto next_col;
53         }
54         return true;
55     next_col:;
56     }
57     for (size_t r = 0; r < BOARD_SIZE; ++r)
58         if (mark(board, r, r) != m)
59             goto other_diagonal;
60     return true;
61 other_diagonal:
62     for (size_t r = 0; r < BOARD_SIZE; ++r)
63         if (
64             mark(board, r, BOARD_SIZE - r - 1) != m
65         )
66             return false;
67     return true;
68 }
69
70 bool is_done(uint32_t board) {
71     return (
72         is_full(board)
73         || has_won(board, 1)
74         || has_won(board, 2)
75     );
76 }
77
78 uint8_t move (
79     uint8_t result, size_t r, size_t c
80 ) {
81     return (result << 4) | (r << 2) | c;
82 }
83
84 bool is_loss(uint8_t move) {
85     return move & 0x10;
86 }
87
88 bool is_draw(uint8_t move) {
89     return move & 0x20;
90 }
91
92 bool is_win(uint8_t move) {
93     return move & 0x40;
94 }
95
96 int player(uint32_t board) {
97     size_t marks[3] = {0};
98     for (size_t r = 0; r < BOARD_SIZE; ++r)
99         for (size_t c = 0; c < BOARD_SIZE; ++c)
100             marks[mark(board, r, c)]++;
101     return (marks[1] <= marks[2]) ? 1 : 2;
102 }
103
104 uint8_t ttt(uint32_t board, uint8_t *best) {
105     assert(!is_done(board));
106
107     if (best[board])
108         return best[board];
109
110     int p = player(board);
111     uint8_t b = 0;
112
113     for (size_t r = 0; r < BOARD_SIZE; ++r)
114         for (size_t c = 0; c < BOARD_SIZE; ++c)
115             if (!mark(board, r, c)) {
116                 uint32_t new_board = place (
117                     board, r, c, p
118                 );
119                 if (has_won(new_board, p))
120                     return best[board] = move(WIN, r, c);
121                 if (is_done(new_board))
122                     return best[board] = move (
123                         DRAW, r, c
124                     );
125                 uint8_t f = ttt(new_board, best);
126                 if (is_loss(f))
127                     return best[board] = move(WIN, r, c);
128                 if (is_draw(f))
129                     b = move(DRAW, r, c);
130                 else if (!b)
131                     b = move(LOSS, r, c);
132             }
133     assert(b);
134     return best[board] = b;
135 }
136
137 int main() {
138     uint8_t *best = calloc (
139         power(3, 16),
140         sizeof(uint8_t)
141     );
142     uint32_t board = 0;
143
144     while (!is_done(board)) {
145         if (player(board) == 1) {
146             uint8_t move = ttt(board, best);
147             size_t r = (move >> 2) & 3, c = move & 3;
148             printf("%zu␣%zu␣\n", r, c);
149             board = place(board, r, c, 1);
150         } else {
151             size_t r, c;
152             scanf("%zu␣%zu", &r, &c);
153             board = place(board, r, c, 2);
154         }
155     }
156
157     free(best);
158     return 0;
159 }

```

## 18 Compiling and Running Programs

This chapter lists some common commands used for compiling and running C/C++ programs.

To compile a program in `a.c` into an executable in file `a` for testing:

```
gcc -std=c23 -Wall -Wpedantic -g a.c -o a
```

To compile a program in `a.c` into an executable in file `a` for fast execution:

```
gcc -std=c23 -Wall -Wpedantic -O3 a.c -o a
```

For C++:

```
g++ -std=c++23 <same as gcc options>
```

## References

- [CLRS22] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, MA, 4th edition, 2022.